

DriveIQ — Account & Email Lifecycle

2026-09-18

What exists today, what is missing, and what it would take to build the full signup and premium email journeys.

What exists today

More groundwork than you'd expect, but only one email actually reaches a user today.

Brevo is wired up and proven — with a single caller. `sendWaitlistCodeByBrevo()` in `functions/src/index.ts` posts to the Brevo API, and the secrets `BREVO_API_KEY`, `BREVO_SENDER_EMAIL` and `BREVO_SENDER_NAME` are already defined and in use. It fires from exactly one place: `requestWaitlistCodeHttp`, when a waitlist user asks to resend their claim code. The sending pipeline works — it just has one job.

Branded templates already exist. `functions/src/waitlistEmail.ts` holds table-based HTML built for Gmail, Outlook and Apple Mail, using the DriveIQ palette. Two builders: `buildWaitlistClaimCodeEmail()` (in use) and `buildWaitlistLaunchEmailTemplate()` — which appears to be **unused**, a launch template written but never wired up. Worth reading before anyone designs new templates; the house style is already set.

Email verification exists, and it is the only thing Firebase sends. `sendVerifyLink()` calls Firebase Auth's built-in `sendEmailVerification()`, using Firebase's default template from Firebase's own infrastructure — not Brevo, and not branded.

What does not exist

Email	Status
Welcome email on signup	Nothing. There is an in-app "account ready" sheet, but no email
Premium / trial started	Nothing
Day-6 "payment is coming" reminder	Nothing
Weekly or usage-based sends	Nothing — no scheduled job, no segmentation
Anything for Apple / Google signups	Nothing at all

That last row is the sharpest. Apple and Google signups skip `sendVerifyLink` entirely, because the provider has already verified the address. So a brand-new user who downloads from the App Store and signs up with Apple currently receives **zero emails, ever** — no verification, no welcome, nothing. Given Sign in with Apple is the most prominent button on the sheet, that is likely the majority path.

Decide this first: we can't currently email anyone

This is the one thing that shapes everything below, and it's easy to miss.

The `users/{uid}` document in Firestore holds `notificationPrefs`, `lineSubscriptions`, `savedFlights`, `savedEvents`, `fcmTokens` and `freeStationSlot`.

It does **not** hold the email address. Or the display name. Or the signup date.

So a scheduled job that wants to "email everyone who signed up this week" or "email free users who use AI chat a lot" has nothing to query. The addresses exist — in Firebase Auth — but not anywhere a Cloud Function can filter on.

Three ways round it

Write the email onto the user doc at signup. Add `email`, `displayName`, `createdAt`, `emailVerified` and `authProvider` to `users/{uid}` when the account is created. Simple, and every scheduled job becomes an ordinary Firestore query. Downside: a second copy of personal data to keep in step and to delete on account deletion — though `deleteAccountHttp` already clears that document, so the deletion path is covered.

Page through Firebase Auth from the job. The Admin SDK's `listUsers()` walks every account. No duplication, but it's a full scan every run, it can't filter or join against usage, and it gets slow as the base grows. Fine for a one-off launch blast, wrong for a weekly job.

Push the address into Brevo at signup and let Brevo own the list. Brevo becomes the source of truth for contacts and segments; the app just tells it when someone signs up, upgrades or cancels. This is what Brevo is for, and it's where the automation, unsubscribe handling and deliverability reporting already live.

Recommendation

Do the **first and third together**: write the fields onto `users/{uid}` so our own jobs can reason about accounts, and sync the contact to Brevo so marketing sends and unsubscribes are handled where they belong. The middle option isn't worth building on.

Whichever way — nothing else on this page can be built until this is settled.

New user signup — the full journey

A fresh install from the App Store, no waitlist code.  = exists,  = to build.

#	Step	Where	Status
1	Downloads, opens app, browses anonymously	App	
2	Hits a gated action (save, alert, AI) → account prompt	App	
3	Signs up: Apple, Google, or email + password	Firebase Auth	
4	Verification email (email signups only)	Firebase Auth	 — unbranded
5	Account fields written to <code>users/{uid}</code>	Cloud Function	
6	Contact synced to Brevo	Cloud Function	
7	In-app "account ready" sheet	App	
8	Welcome email	Brevo	
9	Usage tracked for later segmentation	PostHog	

On step 4 — verification

Zakariye asked for "some type of email auth". It's already there for email signups, but two things are worth knowing.

It's **Firestore's default template**, not ours — generic wording, Firebase branding, and it comes from a Firebase sending domain. It can be customised in the Firebase console (Authentication → Templates) including a custom sender domain, but it can't be built in Brevo, because Firebase generates the one-time verification link.

And it **does not fire for Apple or Google signups**. That's correct — the provider has already verified the address, so there's nothing to verify. But it means "verified in app" can't be the trigger for the welcome email, or Apple users never get one.

On step 8 — the welcome email

Trigger it on account creation, not on verification. Otherwise Apple and Google users are silently excluded — probably most of the base.

First name is easy to personalise where we have it: Apple gives `fullName` on **first authorisation only** (we already capture it and set the display name), Google gives it every time, and email signup asks for it directly. Where it's missing, fall back to a generic greeting rather than "Hi ,".

Content should mirror the in-app account-ready sheet so the two reinforce each other: what a free account gives you (saves, disruption alerts, AI questions, the map), one clear next action, and a quiet line about Premium — not a hard sell in the first email.

Premium and the 7-day trial

#	Step	Where	Status
1	Taps Upgrade → paywall	App	✓
2	Picks annual or monthly, confirms with Apple	StoreKit	✓
3	RevenueCat records the entitlement	RevenueCat	✓
4	In-app "welcome to Premium" walkthrough	App	✓
5	"Your free trial has started" email	Brevo	🔨
6	Day 6: "payment is coming" reminder	Brevo	🔨
7	Cancellation / lapse email	Brevo	🔨

The piece that makes this work: RevenueCat webhooks

Zakariye asked how we'd "keep track of the dates". We shouldn't compute them ourselves — and importantly, **we can't reliably**.

The app only knows about a subscription while it's open. If someone subscribes and doesn't reopen the app for a week, nothing client-side can fire the day-6 reminder. Same for cancellations: Apple never tells the app, and the user may cancel from Settings without opening DriveIQ again.

RevenueCat already knows all of this, and it emits **webhooks** for exactly these events: `INITIAL_PURCHASE`, `TRIAL_STARTED`, `RENEWAL`, `CANCELLATION`, `EXPIRATION`, `BILLING_ISSUE`. We're already using RevenueCat for entitlements, so this is configuration plus one endpoint — not a new integration.

That gives a clean chain: **RevenueCat webhook → Cloud Function → Brevo send**. Server-side, exact, fires whether or not the app is open.

Day 6 specifically

Don't schedule it from the client. Either let Brevo handle it as an automation timed off the trial-start event, or write a `trialEndsAt` date when the trial-start webhook lands and have a daily Cloud Function sweep for trials ending tomorrow. Brevo's automation is less code; the sweep is easier to reason about and test.

One thing to get right

Apple already sends its own trial-ending and receipt emails. Ours should **complement, not duplicate** — lead with what they get from Premium and how to manage the subscription, rather than restating Apple's billing notice. Two near-identical emails on the same day reads as a mistake.

Also worth flagging: the **waitlist free week is a different thing** from the StoreKit 7-day trial. The waitlist week is granted server-side with no payment method attached and simply ends. Don't send "payment will be taken on day 8" to those users — it isn't true for them, and it's the kind of error that produces support tickets and refund requests.

Who sends what

Direct answer to Zakariye's question — "are these emails sent from Firebase?" Partly. Only one is, and it's the one we have least control over.

System	Owns	Sends
Firestore Auth	Accounts, passwords, identity	Verification + password reset, only
Firestore	App data on <code>users/{uid}</code>	Nothing
Cloud Functions	Triggers, scheduled jobs, webhook endpoints	Calls Brevo
Brevo	Contact list, templates, automations, unsubscribes	Every other email
RevenueCat	Subscription state and its events	Nothing — but tells us when to send
PostHog	Feature usage, per-person properties	Nothing — but tells us who to send to

Why this split

Firebase keeps verification and password reset because it mints the one-time links. Those can't move to Brevo. They can be restyled in the Firebase console, and should be — including a custom sender domain so they don't arrive from a firebaseapp.com address.

Brevo owns everything else, including unsubscribe handling and suppression lists. That matters legally as much as practically: marketing email needs a working unsubscribe, and reinventing that in Cloud Functions is a bad trade when Brevo does it already.

PostHog is the source of truth for behaviour, not Firestore. `identifyFirebaseUser()` already sets `$email`, `tier`, `auth_provider`, `signed_in` and `email_verified` on the person, and `track()` logs feature events throughout the app. So "free users who asked the AI more than five questions this week" is a PostHog question. Don't rebuild that in Firestore — it already works, and the data's already flowing.

The one decision here

There are two viable shapes, and it's worth picking deliberately:

Cloud Functions drive the sends — our code decides who gets what and calls Brevo's transactional API. Full control, everything in the repo and reviewable, but we build the scheduling and segmentation ourselves.

Brevo drives them — we sync contacts and events, and marketing builds journeys in Brevo's UI without a deploy. Much faster to iterate on copy and timing; the logic then lives outside the repo.

My take: **transactional stays in code** (welcome, trial started, day 6 — these must be exact and tested), **marketing moves to Brevo** (weekly roundups, win-backs, premium nudges — these want fast iteration and someone non-technical editing them).

Ongoing and usage-based email

Zakariye's example — *"one user consistently uses free features and could be sent an email with a premium CTA"* — is exactly the right shape, and the data for it already exists.

How it would work

PostHog already records every feature event and holds `tier`, `auth_provider` and `$email` per person. A cohort like *free tier, 5+ AI questions in the last 7 days, no paywall view* is a PostHog query today — nothing new to instrument.

From there, two options. PostHog can sync cohorts straight into Brevo as a list, and Brevo sends to it — no code at all. Or a weekly Cloud Function queries PostHog, then calls Brevo per user — more control, more to maintain. Start with the first; move to the second only if it can't express something we need.

Candidate sends

Segment	Email	Why
Free, heavy AI use	Premium CTA	Already hitting the daily cap
Free, saves lots of events	Premium CTA	Wants the full calendar
Signed up, never returned	Re-engagement	Onboarding didn't land
Premium, low usage	Feature spotlight	Churn risk before renewal
Trial ended, didn't convert	Win-back	Timed off the RevenueCat event
Everyone (opt-in)	Weekly what's-on	Only worth it if the content is genuinely good

Two cautions

Frequency. Between disruption push, event reminders and marketing email, a heavy user could be hearing from us several times a day. Cap marketing at roughly one a week and keep it separate from transactional, or people mute everything — including the alerts they actually signed up for.

The weekly roundup is the hardest one to do well. A generic "here's what's on" is easy to ignore and easy to unsubscribe from. It only works if it's genuinely tailored — their saved venues, their area, the lines they follow. That's real work, and I'd put it last rather than first.

Build order

Ordered so each step unlocks the next. Effort is rough and assumes templates are being designed in parallel.

1. Account fields on `users/{uid}` + Brevo contact sync · *small*

Write `email`, `displayName`, `createdAt`, `emailVerified`, `authProvider` on account creation; push the contact to Brevo. Nothing else can be built until this exists. Also decide now whether marketing consent is opt-in at signup — retrofitting consent later is painful.

2. Welcome email · *small*

Triggered on account creation (not verification, so Apple and Google users get it). Reuses the existing Brevo call and the template style already in `waitlistEmail.ts`. This is the first real win and it's cheap.

3. Brand the Firebase verification email · *small, no code*

Firebase console → Authentication → Templates. Wording, branding, and a custom sender domain. Config only, and it's the first email a new email-signup user ever sees.

4. RevenueCat webhook endpoint · *medium*

One Cloud Function receiving `TRIAL_STARTED`, `RENEWAL`, `CANCELLATION`, `EXPIRATION`, `BILLING_ISSUE`. Verify the signature. This unlocks every premium email and is the highest-value piece after the welcome.

5. Trial-started and day-6 emails · *small, once 4 exists*

Trial-started fires straight off the webhook. Day 6 either as a Brevo automation or a daily sweep over `trialEndsAt`.

6. PostHog → Brevo cohort sync · *small, mostly config*

Turns on all the usage-based segments at once.

7. Premium nudges and win-backs · *small each, in Brevo*

Marketing can build and iterate these without a deploy.

8. Weekly roundup · *large — do last*

Only worth it personalised. See the caution above.

Before launch

Steps **1–3** are the minimum for a credible launch — without them a new user gets either an unbranded Firebase email or, if they signed up with Apple, nothing at all. Steps **4–5** should follow quickly: the first person to start a paid trial and hear nothing from us is a support ticket waiting to happen.

And the **Brevo plan upgrade** Zakariye mentioned should be done before launch regardless — the waitlist blast alone will exceed the free tier, and everything here sends through the same account.

Open decisions

These need answers before step 1, because each one changes what gets built.

1. **Where does contact data live?** Firestore, Brevo, or both. Recommendation above is both — but it's a call, not a given.
2. **Is marketing consent opt-in at signup?** Much easier to add now than to retrofit. Affects the signup UI as well as the backend.
3. **Who owns the sends — code or Brevo?** Suggested split: transactional in code, marketing in Brevo.
4. **Do we personalise by first name?** We have it for most users but not all; needs a decided fallback.
5. **What's the frequency cap on marketing email?** Suggested: one a week, separate from transactional and push.

The App Store release check

Zakariye flagged this and it's the one item here with no undo, so treating it separately.

I could not verify it from the codebase. `eas.json` sets only `ascAppId` — there's no release-type field. Manual vs automatic release is an **App Store Connect setting**, not something in the repo, so it has to be checked by hand.

Where to look: App Store Connect → DriveIQ → the version awaiting review → scroll to **"Release This Version"** → confirm **"Manually release this version"** is selected, not "Automatically release this version".

It's editable while the version is in review, so it can be fixed now — but not after approval fires an automatic release. Worth checking today rather than on approval day.

Summary

The pipeline exists and works; it just has one caller. The templates exist and are already branded. The usage data exists in PostHog. The subscription events exist in RevenueCat.

What's missing is the wiring between them — plus one small gap that blocks all of it: **the user document doesn't store an email address**. Fix that, and the first three steps are days rather than weeks.